ARTIFICIAL INTELLIGENCE (Q) – APRIL 2026
KECERDASAN TIRUAN (Q) – APRIL 2026

Particle Swarm Optimization

Optimasi Kawanan Partikel

Mohammad Iqbal, S.Si., M.Si., Ph. D.

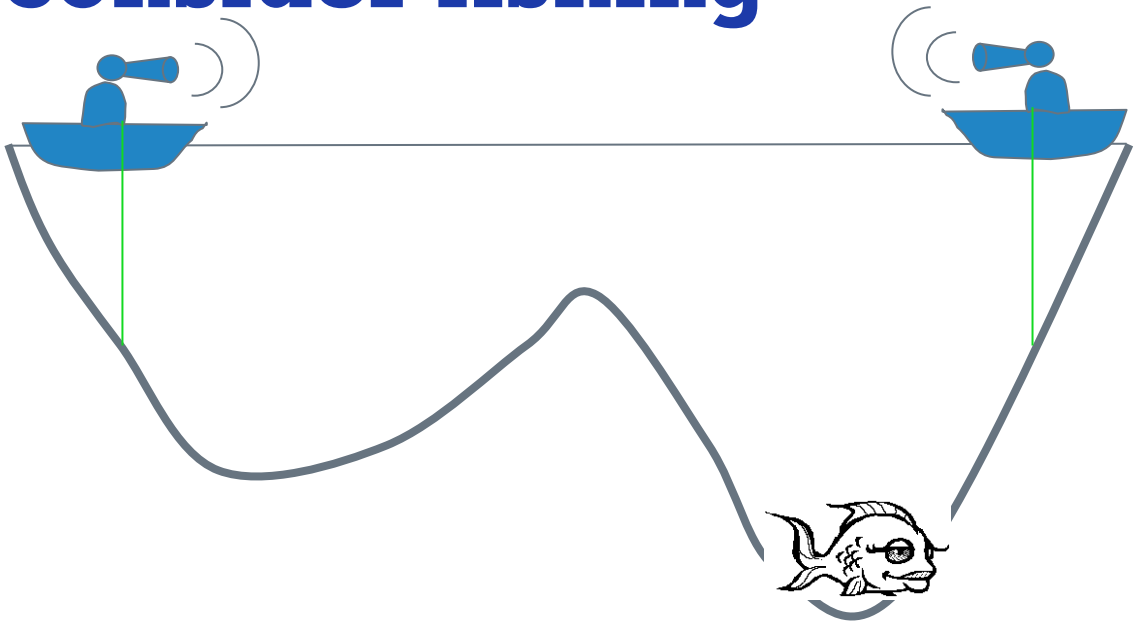
OUTLINES

- Illustration
- Basic Idea

OUTLINES

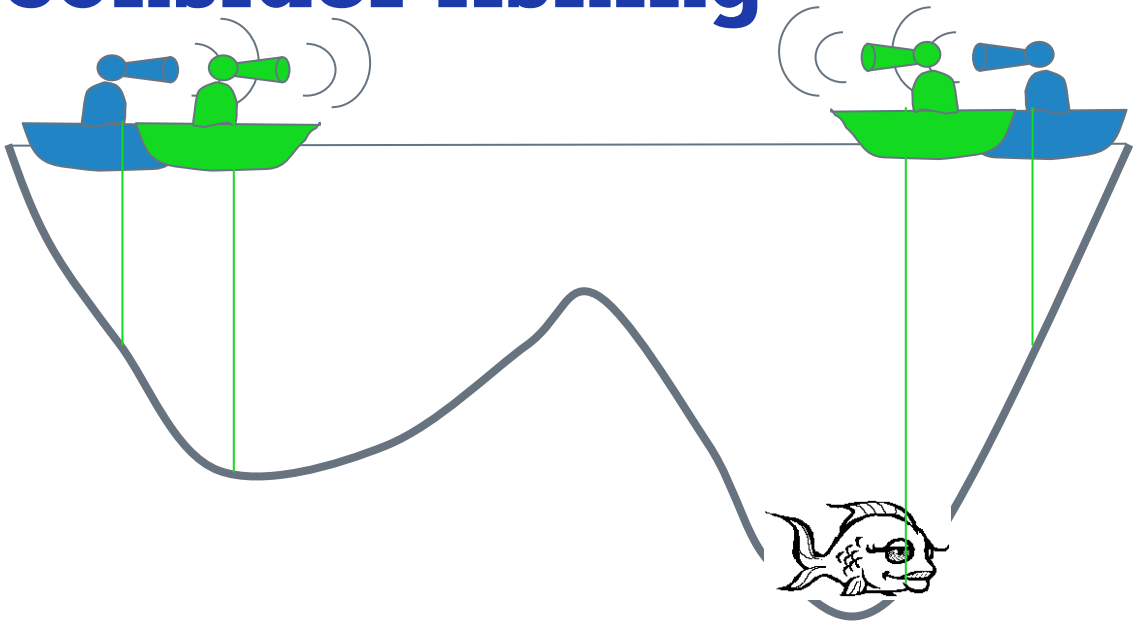
Illustration

Let us consider fishing



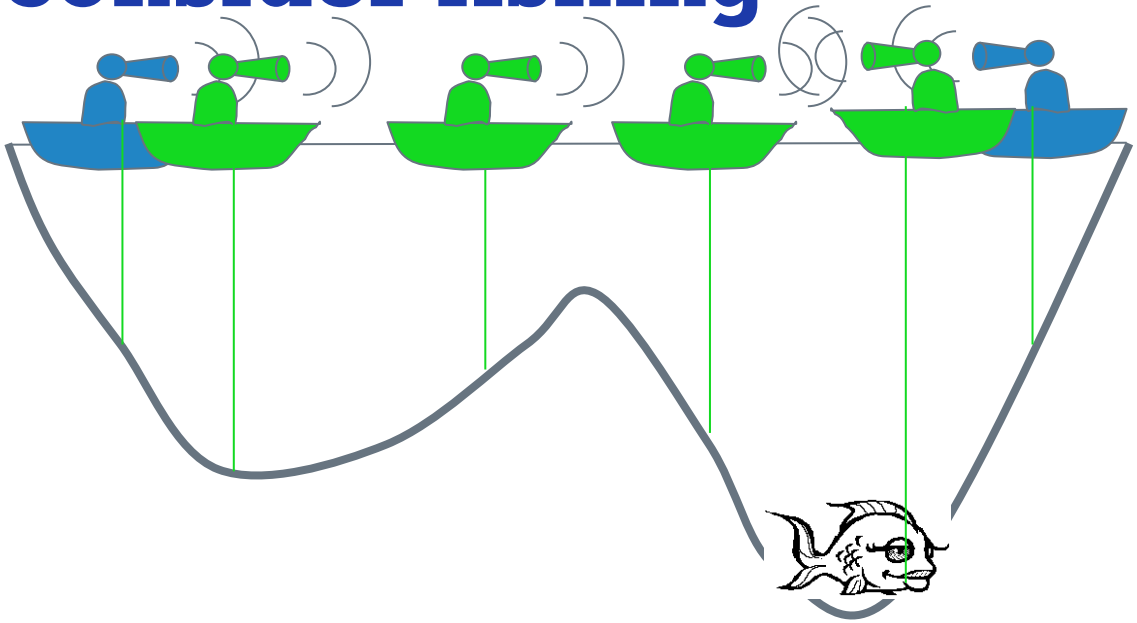
- Big fish is difficult to catch as it hides in the deepest part of the pond.
- At each time, each fisherman communicates to the other how deep the pond from him.

Let us consider fishing



- Assume the better position is Fisherman 2
- Accordingly, fisherman 1 wants to go to Fisherman 2
- But, his position is getting worse and needs to compromise (slower than before)

Let us consider fishing



- Then, Fisherman 1 escapes from the local minimum

Main features on PSO

- A ***position***, a ***velocity*** (or, more precisely an operator which can be applied to a position in order to modify it)
- The ability to *exchange information* with its neighbours.
- The ability to *memorize* a previous position.
- The ability to *use information* to make a decision.

PSO origins

- Developed by **Jim Kennedy,** Bureau of Labor Statistics, U.S. Department of Labor and **Russ Eberhart,** Purdue University at 1995
- A concept for optimizing nonlinear functions using particle swarm methodology .

OUTLINES

Basic Idea

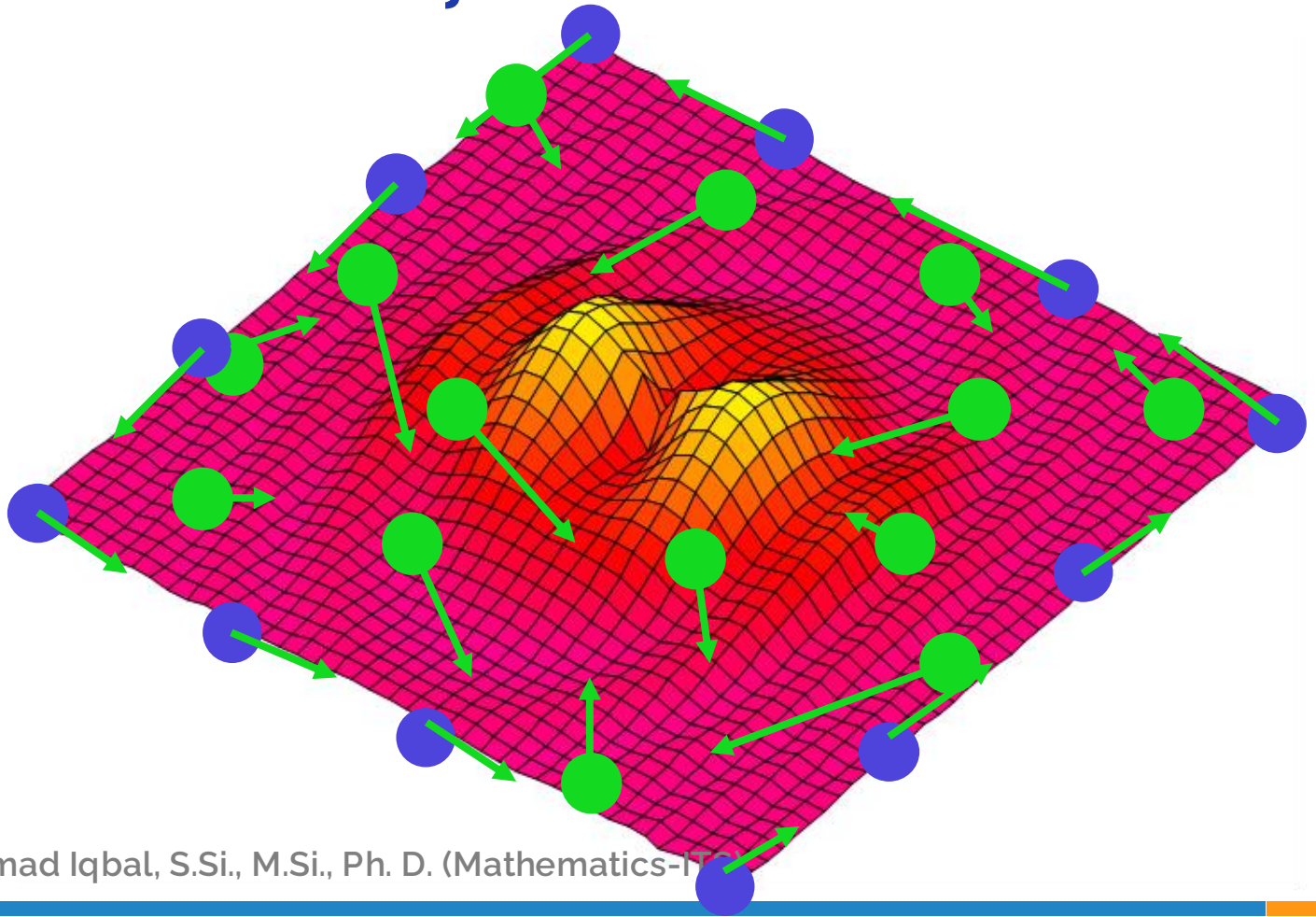
Basic Idea on PSO

- Each particle is ***searching*** for the **optimum.**
- Each particle is ***moving*** and hence has a ***velocity***.
- Each particle ***remembers*** the **position** it was in where it had its best result so far (its ***personal best***)
- *But this would not be much good on its own; particles need help in figuring out where to search.*

Basic Idea on PSO…

- The particles in **the swarm *co-operate***.
- They **exchange information** about what they've discovered in the places they have visited.
- The co-operation is very simple.
- In basic PSO it is like this:
  - A particle has a ***neighbourhood*** associated with it.
  - A particle knows the **fitnesses** of those in its **neighbourhood**, and uses the ***position* of the one with best fitness.**
  - This position is simply used to **adjust the particle's velocity.**

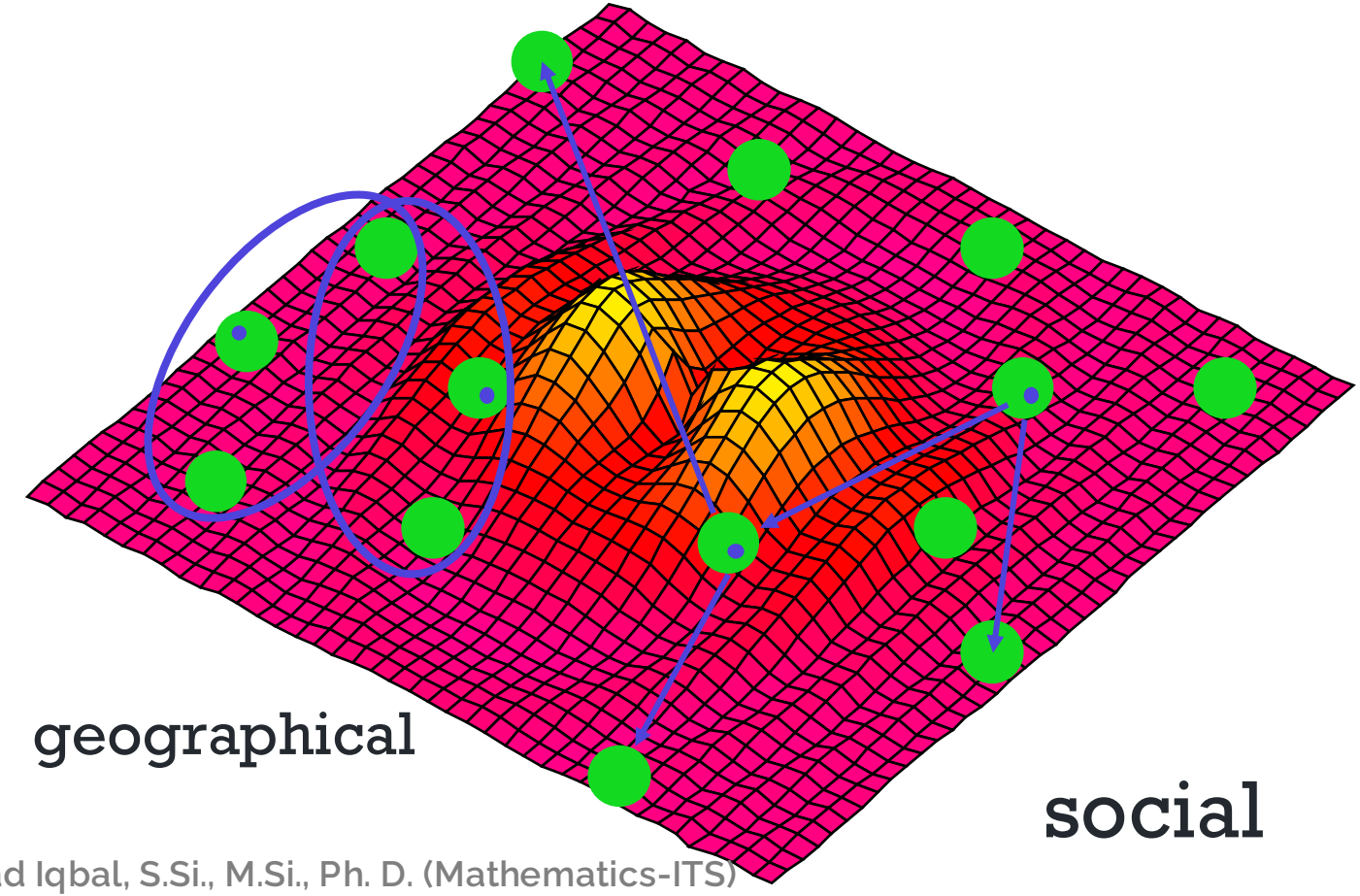
Initialization, Position & Velocities



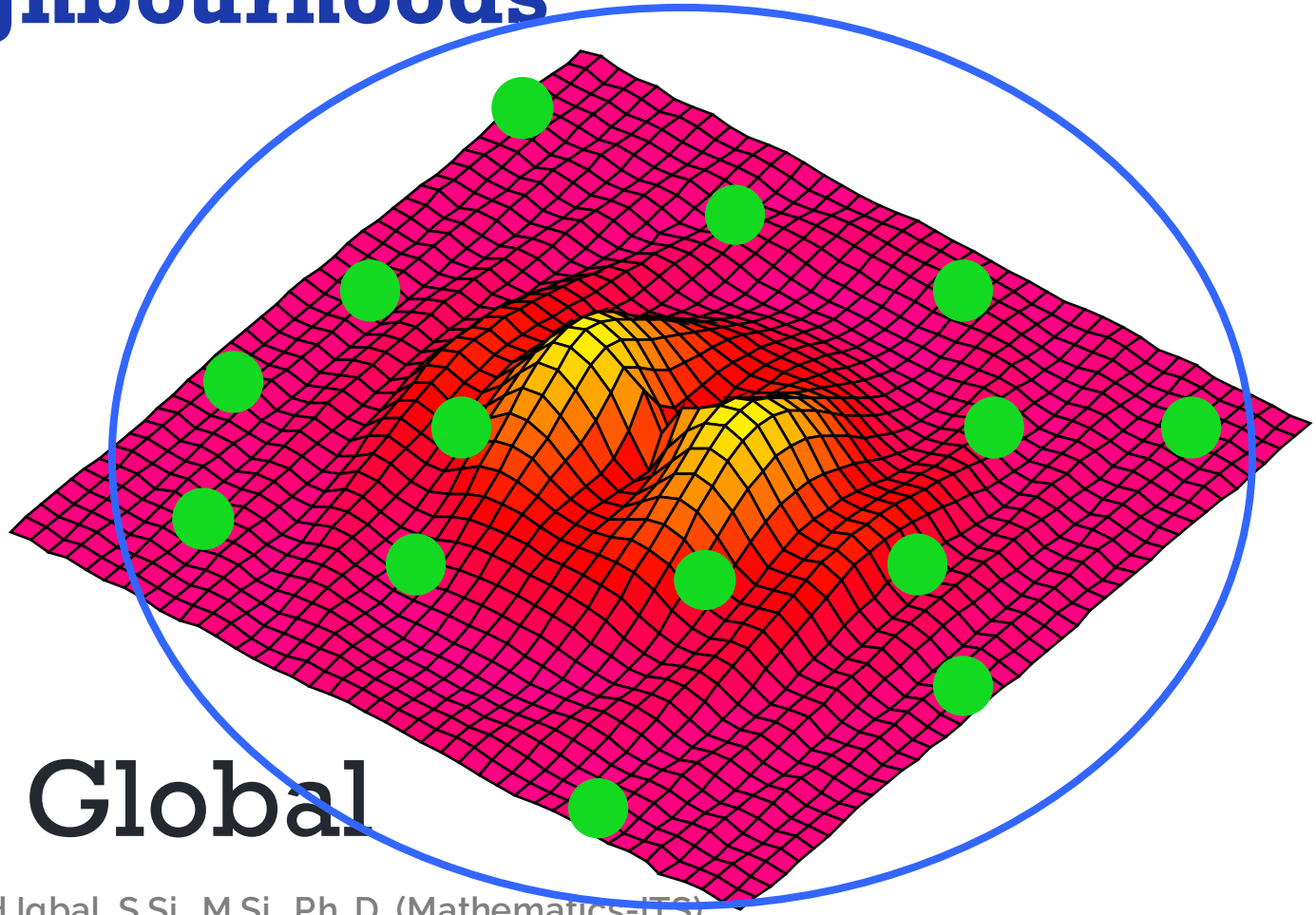
What a Particle does?

- In each timestep, a particle has to move to a new position. It does this by adjusting its *velocity*.
  - *The adjustment is essentially this:*
  - *The current velocity PLUS.*
  - *A weighted random portion in the direction of its personal best PLUS.*
  - *A weighted random portion in the direction of the neighbourhood best.*
- *Having worked out a new velocity, its position is simply its old position plus the new velocity.*

Neighbourhoods

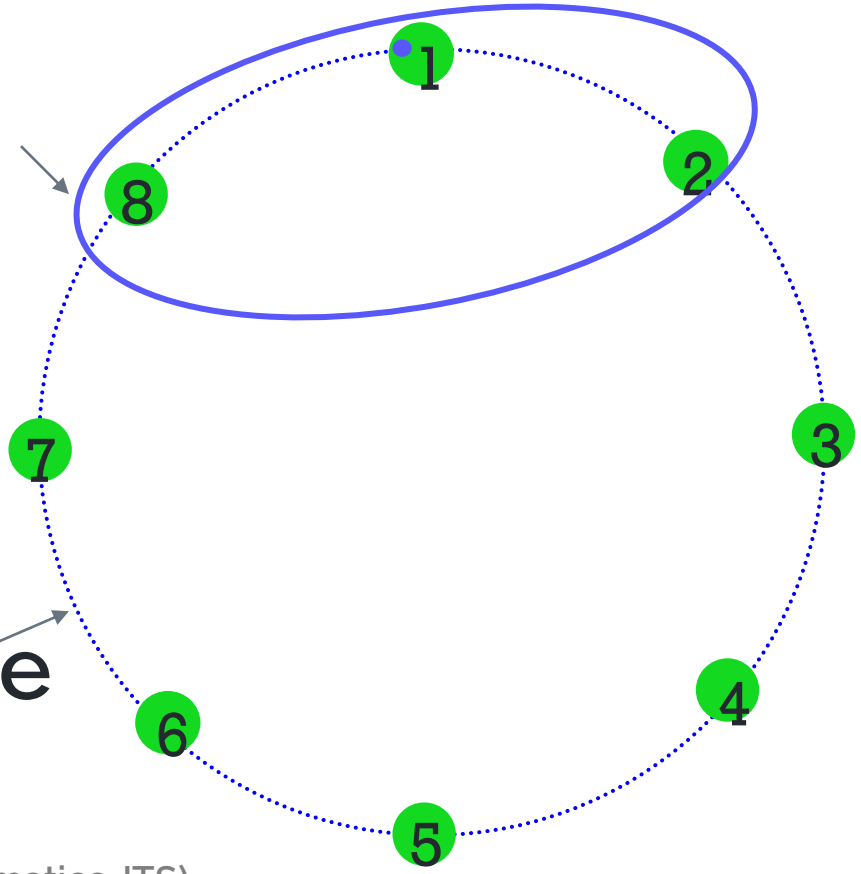


Neighbourhoods



The Circular Neighbourhoods

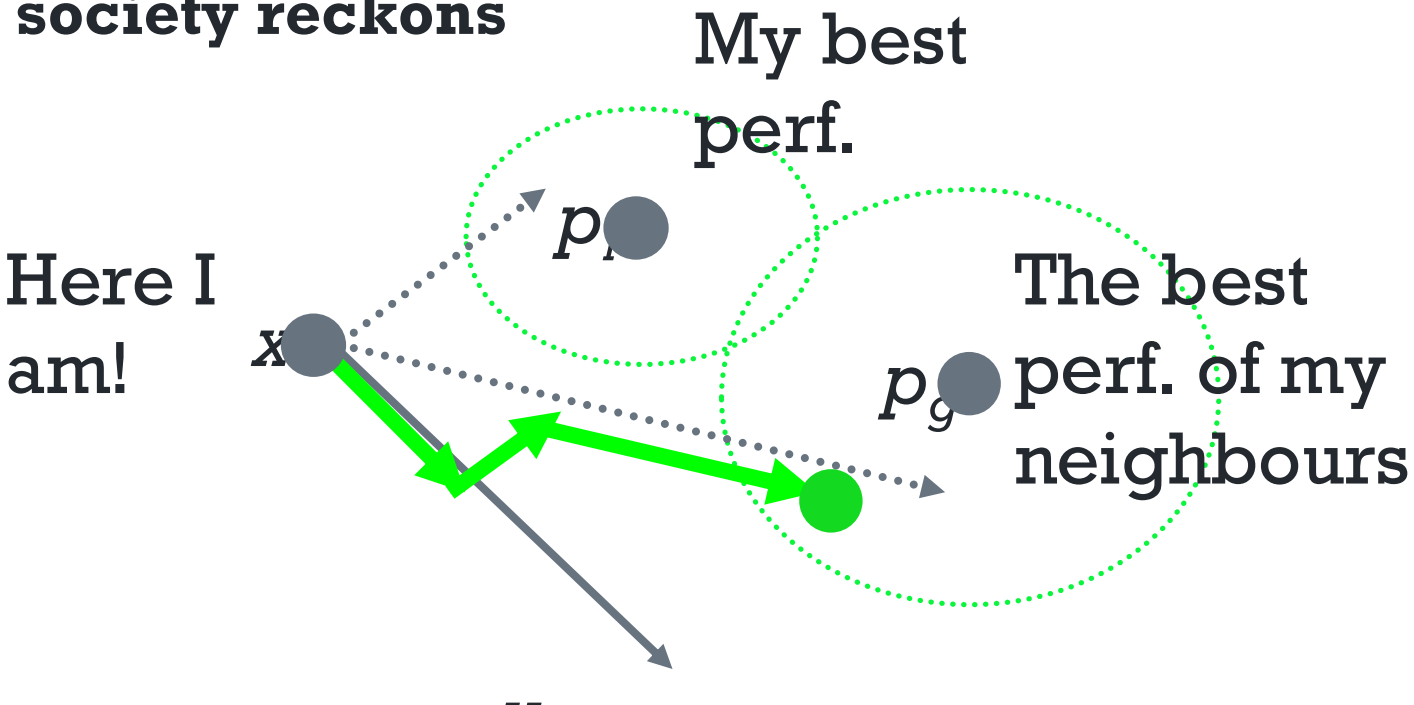
Particle 1's 3-neighbourhood



Virtual circle

Compromise…

Particles Adjust their positions according to a ``Psychosocial compromise'' between what an **individual is comfortable** with, and what **society reckons**



OUTLINES

Pseudocode

momentum

cognitive component

social component

$$\vec{v}_i \leftarrow \vec{v}_i + \boxed{\vec{\varphi}_1 \otimes (\vec{p}_i - \vec{x}_i)} + \boxed{\vec{\varphi}_2 \otimes (\vec{p}_g - \vec{x}_i)}$$

$$\vec{x}_i \leftarrow \vec{x}_i + \vec{v}_i$$

Velocity $\vec{v}_i$ (which denotes the amount of change) of the *i*-th particle is determined by three components:

- **momentum** – previous velocity term to carry the particle in the direction it has travelled so far;
- **cognitive** component – tendency to return to the best position visited so far;
- **social** component – tendency to be attracted towards the best position found in its neighbourhood.

Neighbourhood topologies can be used to control information propagation between particles, e.g., ring, star, or von Neumann. ***lbest*** and ***gbest*** PSOs.

PSO Algorithm

```
Equation (a)
v[] = c0 *v[]
      + c1 * rand() * (pbest[] - present[])
      + c2 * rand() * (gbest[] - present[])
   (in the original method, c0=1, but many
      researchers now play with this parameter)

Equation (b)
present[] = present[] + v[]
```

Inertia weight

The $\vec{p}_i$ and $\vec{p}_g$ can be collapsed into a single term $\vec{p}$ without losing any information:

$$\vec{v}_i \leftarrow \vec{v}_i + \vec{\varphi} \otimes (\vec{p} - \vec{x}_i)$$

$$\vec{x}_i \leftarrow \vec{x}_i + \vec{v}_i$$

where $\vec{\varphi} = \vec{\varphi}_1 + \vec{\varphi}_2$ and $\vec{p} = \dfrac{\vec{\varphi}_1 \otimes \vec{p}_i + \vec{\varphi}_2 \otimes \vec{p}_g}{\vec{\varphi}_1 + \vec{\varphi}_2}$

$\vec{p}$ represents the weighted average of $\vec{p}_i$ and $\vec{p}_g$. Note that the division operator is a point-wise vector division.

Since the velocity term tends to keep the particle moving in the same direction as of its previous flight, a coefficient inertia weight, ***w***, can be used to control this influence:

$$\vec{v}_i \leftarrow \mathbf{w}\ \vec{v}_i + \vec{\varphi}_1 \otimes (\vec{p}_i - \vec{x}_i) + \vec{\varphi}_2 \otimes (\vec{p}_g - \vec{x}_i)$$

The inertia weighted PSO can converge under certain conditions without using **Vmax**.

PSO Algorithm

```
For each particle
      Initialize particle
  END

  Do
      For each particle
          Calculate fitness value
          If the fitness value is better than its peronal best
            set current value as the new pBest
      End

      Choose the particle with the best fitness value of all as
  gBest
      For each particle
          Calculate particle velocity according equation (a)
          Update particle position according equation (b)
      End
  While maximum iterations or minimum error criteria is not attained
```

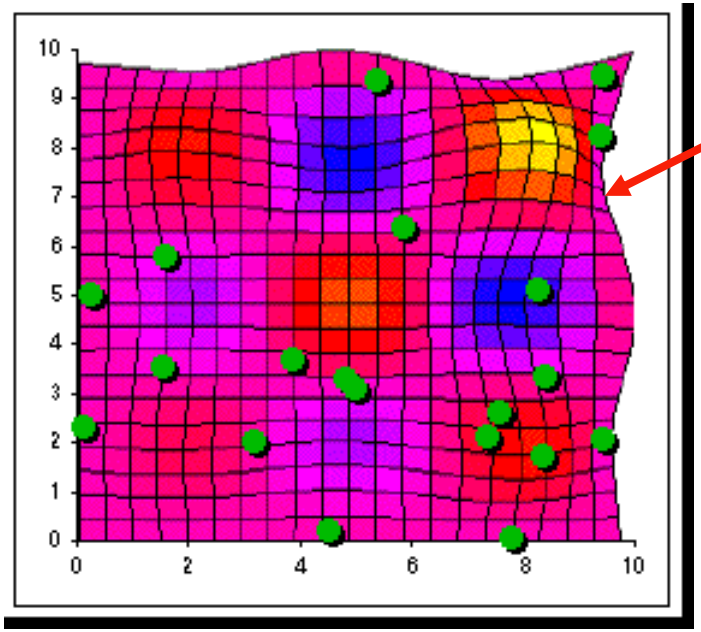
PSO Algorithm

- Particles' velocities on each dimension are clamped to a maximum velocity Vmax.
- If the sum of accelerations would cause the velocity on that dimension to exceed Vmax, which is a parameter specified by the user.
- Then the velocity on that dimension is limited to Vmax.

OUTLINES

Example

Example



Global optimum

Parameters

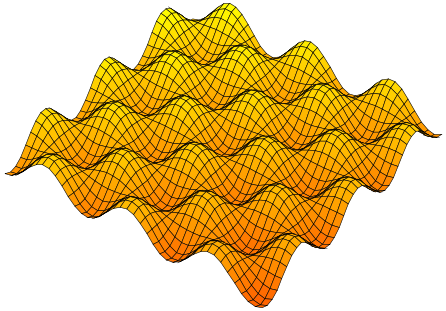
- Number of particles
- C1 (importance of personal best)
- C2 (importance of neighbourhood best)

Parameters

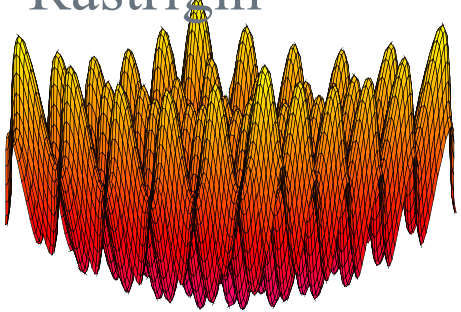
- Number of particles (10—50) are reported as usually sufficient.
- C1 (importance of personal best)
- C2 (importance of neighbourhood best)
- Usually C1+C2 = 4.
- No good reason other than empiricism
- Vmax – too low, too slow; too high, too unstable.

Fitness functions

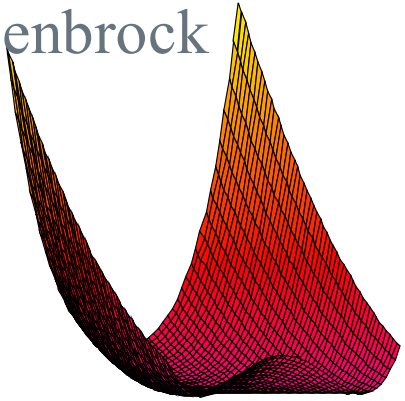
Griewank



Rastrigin



Rosenbrock



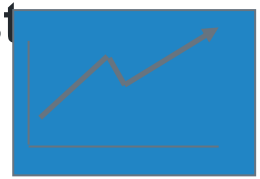
Results

30D function	PSO Type 1"	Evolutionary algo.(Angeline 98)
Griewank [±300]	0.003944	0.4033
Rastrigin [±5]	82.95618	46.4689
Rosenbrock [±10]	50.193877	1610.359

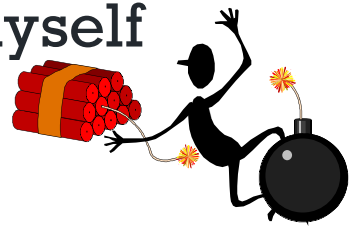
Adaptive swarm size

There has been enough improvement

although I'm the worst



I try to kill myself

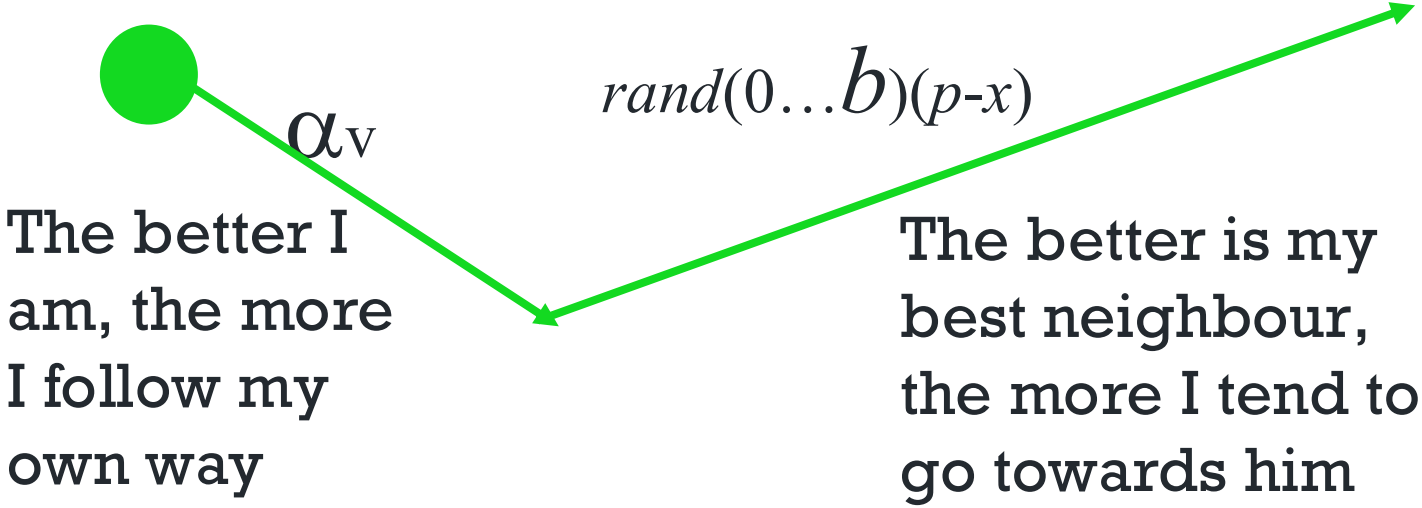


I'm the best

but there has been not enough improvement

I try to generate a new particle

Adaptive parameters



Discussion...

Q n A